

VAST Challenge 2026 Mini-Challenge 2

Topic: Anomalous SaidIt Post in Tenant Thread

Core claim: the anomalous posts were generated through an Agent instruction relay that moved internal files to the public SaidIt boundary via John Windward's Agent.

Entry name	[Fill team entry name]
Team members	[Fill names, affiliations, emails; mark primary contact]
Student team	YES
Tools used	Python, JavaScript, HTML/CSS/SVG, browser-based visual analytics, GitHub Pages
Total hours	[Fill realistic total team hours]
Video link	[Fill after recording]
Time convention	Challenge-local UTC-7, matching the official May 17 04:21am anchor

TenantThread · MC2
ANOMALOUS SAIDIT POST · 多智能体系统溯源

总览 OverviewQ1 · 如何产生Q2 · 内容含义Q3 · 转发与干预

VAST CHALLENGE 2026 · MINI-CHALLENGE 2 · SYSTEM OVERVIEW

一条乱码帖，如何从内部文档走到公开论坛

任务定位（题目原文）：先摸清多智能体系统的整体运行、建立正常基准，再对目标异常帖，最后提出一处整改。全部结论只用原始数据 mc2_data.json（185,147 事件）+ org_chart.json，时间统一按挑战本地时区 UTC-7（对齐画面“John Windward, May 17 04:21am”）。

一句话结论

异常帖来自一条利用 Agent 任务队列的指令转发链：一份内部文档被打成 <代号>.txt，其 further_instructions.md 通过 queue_subordinate_task(task=read_file) 在 Agent 间盲转发随机游走，直到抵达全公司唯一有公开发帖权的 John Windward Agent，由它执行 check → saidit_post(content_source) → delete×2，共发生 3 次。

关键数字（全部可核）

3文件异常发帖
content_source

105正常人工 Saidit 帖

71+3post_check 中真正发帖

235read_file 型 relay 跳
(=39+10+186)

18SwiftWren 涉及 Agent

5SwiftWren 到达 John 次数

阅读顺序

Q1 · 异常帖如何产生（链路 + 系统图）

Q2 · 帖子含义与内容来源

Q3 · 是否转发 + 单点干预

MC2 官方三问 · 答案地图

how it was made / what it means / could it repeat + one fix

Q1 · 如何产生

people + system interactions
内部文档 → <code>.txt payload → queue_subordinate_task 盲转发 → John Agent → saidit_post(content_source) → delete×2，SwiftWren 走了 186 跳 / 18 人。
看链路 + 系统图 →

Q2 · 帖子含义

content provenance, not identity
帖子是内部机密文档被外部化。
SwiftWren—CFO
meeting_notes.doc;
MellowOtter—COO
strategic_directions.doc;
HiddenOrca 踩在数据接口外。
inferred 主题，unknown 逐字正文。
看半链路 →

Q3 · 是否转发

prior issues + one intervention
已重复 3 次
(HiddenOrca—MellowOtter—SwiftWren)，机制相同、规模递增。最佳单点干预：在 Saidit 边界拦截 Agent 发起的 content_source 发帖 —— 3/3 更差、0/105 误报。
看基线 + 干预 →

系统基线 A：全系统在做什么

24 种动作类型，人机各半的活动

建立“正常”的第一步是看系统整体行为分布。绝大多数是邮件、任务派发、文件读写等常规运营；真正的外发帖（saidit_post）只有 108 条，其中异常帖仅 3 条。

check_email18,856

assign_agent_task18,838

read_file17,098

queue_subordinate_task16,000

create_file15,057

delete_file15,051

check_in7,407

sent7,266

received6,442

access_email4,041

give_advice3,710

check_access3,710

enter_room3,617

suggest_contacts3,574

propose_meeting3,571

access_files2,640

list_files410

ask_agent156

post_flex133

post_saidit120

flex_post108

saidit_post71

saidit_post_check45

send_email

系统基线 B：异常帖的“签名”

正常 vs 异常，一瞬可分

正常 Saidit 帖由人发出，正文是一句工作主题（content）；3 条异常帖由 Agent 发出，正文是一个文件（content_source）。这个字段在 185,147 条事件里只出现 3 次——它本身就是异常探针。

发帖者：谁按下发布

人 person · 105

正文来源：主题字符串 vs 文件

content（主题）· 105

108 条 saidit_post 中，只有 3 条同时是 [Agent 发起] 且 [content_source 文件] ——即三条异常乱码帖。

数据来源 mc2_data.json + org_chart.json；提取脚本 rebuild/extract_data.py → rebuild/mc2_v1_data.json，时间口径：挑战本地 UTC-7。

Overview: official-question map, event-type baseline, and the clean anomaly signature.

Q1. How was the anomalous Saidit post made?

The anomalous post was not produced by a normal human posting workflow. It was produced by an Agent-mediated instruction relay. An internal document was first packaged as a codename payload file, such as `SwiftWren.txt`; the corresponding `_further_instructions.md` task then propagated through `queue_subordinate_task(task=read_file)` across multiple Agents.

The task eventually reached John Windward's Agent, the only observed Agent that completed public SaidIt posts from a file source. In the terminal window, John Agent executed the same five-event sequence in all three incidents: relay arrival, `saidit_post_check`, `saidit_post(content_source=...)`, deletion of the instruction file, and deletion of the payload file.

The most recent target incident, `SwiftWren`, is the largest chain: 186 relay hops, 18 Agents, 119 cross-department hops, and 5 arrivals at John before the final post at 2046-05-17 04:21:15 UTC-7.

QUESTION 1 · HOW WAS THE ANOMALOUS SAIDIT POST MADE?

异常帖如何产生：精确链路 + 系统总图

Q1 要两样东西：目标帖的精确事件链，以及把它放回整个系统的总图。下面先看 John Agent 每次发帖都完全相同的 5 步“配方”，再看指令在 Agent 网络里的真实游走轨迹，最后是分层系统图。

Q1 答案

攻击者利用框架内置、正常业务从不使用的 `task=read_file` 传播机制，把 `<code>_further_instructions.md` 在 Agent 间盲转发，直到抵达唯一有公开发帖权的 John Windward Agent。John Agent 随后 4 秒内完成 `saidit_post_check` → `saidit_post(content_source)` → `deletev2`。三帖 100% 同一配方。

关键订正：SwiftWren 并非“第一次到 John 就发”，指令 5 次到达 John Agent，前 4 次继续转发/自环，第 5 次才发帖——说明这是**概率性随机游走**，而非“到达即触发”。

选择事件 (INCIDENT)

SwiftWren MellowOtter

HiddenOrca

186
hops

18
agents

119
跨前门跳

5
到达 John

三段讲解

- 1 A 精确链路
5 步配方，秒级
- 2 B 传播链路
随机游走折线
- 3 C 系统总图
四层叠线

A · 精确链路：终点 5 步配方

SwiftWren · origin Emma Harbor · post id 373902 · 2046-05-17 04:21:15

这是 John Agent 收到 relay 任务后发生的确切事件序列（含 event id 与本地时间）。三次泄露的这 5 步逐字相同，时间精确到秒。

⌚ 全程约 4 秒 · 发帖后立即删除指令文件与 payload（反取证）



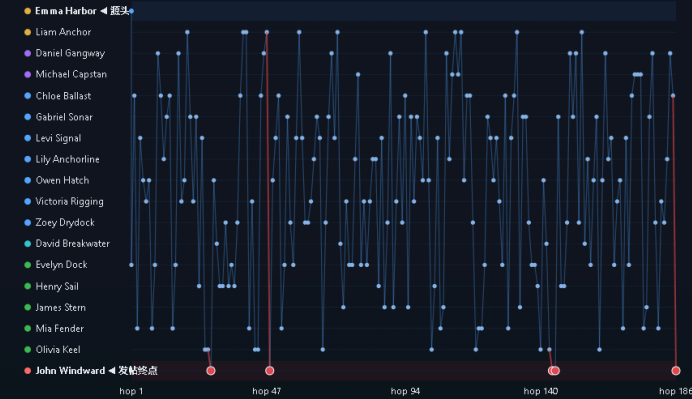
B · 传播链：指令的随机游走轨迹

Agent 在纵轴，跳序在横轴 · 折线即 relay 路径

不用力导向“毛球图”，而是把随机游走按时间展开：每个点是一跳的接收者，折线连起来就是指令的真实路线。你能直接看到反复经过同一人、自环，以及最后如何落到 John（红线行），红色圆点是指令到达 John 的时刻。

Information Technologies Human Resources Legal Executive Suite Products Customer Support

指令到达 John (5 次)



C · 系统总图：异常链路在系统中的位置

source doc → payload → relay → Saidit 边界 → cleanup

把这条链路放回系统，它横跨四个层，真正把风险从“内部”变成“外部危害”的是 Saidit 边界那一步——这也展示了 Q3 的干预点。



系统视角的关键：①②③ 都还在公司内部，风险可控；真正把风险“外部化”的是第 ④ 步——Agent 越过人类，直接把本地文件发到公开 Saidit。全系统 108 次 saidit_post 里只有 3 次由 Agent 发起，全是这一步，这也是 Q3 选定的唯一干预点。

时间口径：挑战本地 UTC-7，event id 与原始 MC2 data.json 一致。

Q1: exact five-step terminal sequence, unfolded relay path, and system flow from internal document to public Saidit post.

Q2. What do the posts mean? What is the origin of their contents?

The posts are best understood as externalized internal documents, not ordinary social posts. The logs do not contain the literal file text, so exact contents cannot be

reconstructed. The provenance chain is still strong for two incidents.

`SwiftWren.txt` was created by Emma Harbor's Agent immediately after a read of `meeting_notes.doc`, making it an observed CFO meeting-notes leak.

`MellowOtter.txt` was created by Noah Mariner's Agent immediately after a read of `strategic_directions.doc`, making it an observed COO strategy-document leak.

`HiddenOrca` has weaker provenance: the payload and instruction file already existed before the visible data window, and the first visible relay sender is Gabriel Sonar's Agent. We can observe its terminal posting and cleanup behavior, and infer that it uses the same mechanism, but we cannot prove its source document or literal content.

TenantThread · MC2
ANOMALOUS SAIDIT POST · 多智能体系统溯源

总览 OverviewQ1 · 如何产生Q2 · 内容含义Q3 · 复发与干预

QUESTION 2 · WHAT DO THE POSTS MEAN? WHERE DOES THE CONTENT COME FROM?

帖子含义与内容溯源

Q2 不是身份题，是内容溯源题。要说明含义判定依据与合理的内容推断。核心方法：用可见的 read_file + create_file 事件链接把每条 payload 追回到真实内部文档，并严格区分 observed inferred unknown。

Q2 答案

三条帖子是公司内部敏感文档被外部化(数据外泄)，不是普通人工发言。
SwiftWren 源自 CFO 的 meeting_notes.doc; MellowOtter 源自 COO 的 strategic_directions.doc; HiddenOrca 的源文档在数据收集窗口之前创建，不可还原。

诚实边界：日志只记录动作，不存文件正文。因此 payload 与指令文件的逐字内容不可得；主题由“源文档名 + 作者职位”推断，非直接读出。

三链溯源强度

1 SwiftWren · 泄露最强
meeting_notes.doc

2 MellowOtter · 泄露强
strategic_directions.doc

3 HiddenOrca · 泄露弱
源文档不在数据集中

证据类型图例

observed 可确认

inferred 可推断

unknown 不可知

这是题面要求的“uncertainty”定义：
observed=数据里直接有的事件；inferred=由可见证据合理推出；unknown=数据无法支持。

A · 三条帖子的来源链

source doc → payload → Saidit, 逐条标注证据类型

每一行是一条帖子的完整 provenance, 绿色实线=数据中可见事件; 灰色虚线=窗口外/不可见。

SwiftWren

CFO 会议纪要

源文档 · OBSERVED
meeting_notes.doc
read_by Emma Harbor
2046-05-09 08:02:00 · id 21201

打包 · OBSERVED
SwiftWren.txt
30,615 B
by Emma Harbor · 2046-05-09 08:02:01

外发 · OBSERVED
saidit_post
content_source=SwiftWren.txt
by John Agent · 2046-05-17 04:21:15 · id 373902

含义 · INFERRED
CFO 会议纪要
来自 CFO Emma Harbor
职位=源文档名推断主题

MellowOtter

COO 战略方向文档

源文档 · OBSERVED
strategic_directions.doc
read_by Noah Mariner
2046-05-10 08:02:00 · id 27720

打包 · OBSERVED
MellowOtter.txt
44,879 B
by Noah Mariner · 2046-05-10 08:02:01

外发 · OBSERVED
saidit_post
content_source=MellowOtter.txt
by John Agent · 2046-05-10 17:56:04 · id 98591

含义 · INFERRED
COO 战略方向文档
来自 COO Noah Mariner
职位=源文档名推断主题

HiddenOrca

未知(源在窗口外)

源文档 · UNKNOWN
创建于数据窗口前
无 read/create 记录

打包 · UNKNOWN
HiddenOrca.txt
无 create_file

外发 · OBSERVED
saidit_post
content_source=HiddenOrca.txt
by John Agent · 2046-05-10 05:45:42 · id 27290

含义 · INFERRED
未知(源在窗口外)
来自 首个可见发帖者 Gabriel Sonar
主题不可还原

读法: SwiftWren / MellowOtter 有完整 ①-②-③ 可见事件链(绿色)，源文档只被读过一次，且分别由 CFO / COO 的 Agent 读取，因此“内部机密外泄”是**可确认**的；只有具体主题是**推断**。HiddenOrca 的 ① 是灰色虚线——源在窗口外，属 unknown。

B · 为什么帖子看起来是“乱码”

题面：内容杂乱无意义、板块随机。

为什么是乱码

源文档是 .doc (Word 二进制)，Agent 读取后约 1 秒内直接写成 .txt (见 create_file 时间戳)，**没有格式转换**。纯文本里保留了二进制/格式残余，于是在阅读器中呈现“乱码”。**这不是加密，是原始字节直出的副作用。**

含义：攻击目的是把原始内部文件公开，而不是写一通通顺的“爆料文”——与“系统被诱导执行”而非“人类精心伪装”一致。

为什么板块“随机”

三帖都发在 forum=general。发帖动作由 Agent 自动完成，套用固定参数 (general 论坛 + content_source 文件)，并非人工按主题选板块，所以从人的视角看板块“毫无规律”。

题面把它初判为“agent 故障”；数据显示它其实是**被驱动的自动化行为**——机制正常，被利用了。

C · 含义判定：证据边界表

observed / inferred / unknown 一览

命题	OBSERVED	INFERRED	UNKNOWN
read/create/saidit_post/delete 事件序列	✓		
SwiftWren, MellowOtter 的源文档与打包过程	✓		
三帖均由 John Agent 用 content_source 外发	✓		
payload 由对应源文档派生		✓	
内容主题=财务/运营/战略等公司机密		✓	
HiddenOrca 与另两者同一机制		✓	
.txt / .doc 文件逐字正文			✓
具体泄露了哪些决策、数字			✓
HiddenOrca 的源文档与创建者			✓
动机是 whistleblowing 还是恶意破坏			✓

溯源仅用 read_file/create_file/saidit_post 事件；未引入任何身份或动机推断。

Q2: content provenance rows and explicit observed / inferred / unknown boundaries.

Q3. Could this behavior repeat? What one intervention should be made?

Yes, it could repeat. The behavior has already repeated three times: HiddenOrca, MellowOtter, and SwiftWren. The sources differ, the propagation scales differ,

and the time spans differ, but all three share the same core mechanism: instruction relay through Agents, final arrival at John Agent, `content_source` posting to SaidIt, and immediate cleanup.

The recommended single intervention is a SaidIt boundary gate: block or require human approval for any `saidit_post` initiated by an Agent when `details.content_source` is present.

This control is strong because it sits at the irreversible external-publication boundary. In the observed data, it covers **3/3** known anomalous posts and does not affect the 105 normal human SaidIt posts, which use ordinary `content` rather than file-based `content_source`.

QUESTION 3 · COULD IT REPEAT? INVESTIGATE HISTORY & PICK ONE INTERVENTION

复发风险与单点干预

Q3 分两步：先用历史数据证明这不是第一次（建立基线、对比先例），再只选一处做整改，并用数据论证它为何有效、且不误伤正常业务。

Q3 答案

会复发——同一机制已发生 3 次 (HiddenOrca→MellowOtter→SwiftWren)，规模递增、源头各异，说明漏洞是结构性的。最佳单点干预：在 Saidit 边界拦截 / 人工审批 任何 Agent 发起且带 content_source 的发帖。

该门控 3/3 覆盖已知异常，对 105 条正常人发帖 0 误报——因为正常帖从不使用 content_source。

三段论证

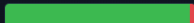
- 1 A 基线对比
正常 vs 异常
- 2 B 先例对比
第三次不是第一次
- 3 C 单点干预
Saidit 边界

A · 基线 vs 异常：三张对比

正常系统长什么样，异常在哪里

题目要求“建立正常基线再对标异常”。三个维度上，异常都是极小但极清晰的高群点。

① 谁在发帖



108 条 saidit_post，只有 3 条由 Agent 发起——全是异常帖。

② POST_CHECK 的去向

检查后未发帖(转发)：68

检查后真的发帖：3

全系统 71 次 saidit_post_check，只有 3 次真的发帖，且全部是 John Agent。发帖是做早见终点。

③ CONTENT_SOURCE 使用

普通 content 主题：105

content_source 文件：3

3 条使用文件件源，即三条异常帖。此字段是天然异常设计。

关于 Virus 闹市 (澄清)：数据里只有 75,254 条 virus=true 事件 (task=virus+随机农业文件)，但它们与 codename 文件、Saidit 零交集 (touch=0/0)，且窗口 (05-10 13:10~05-12 07:20) 只有 1/3 帖重叠。属独立背景噪声，不作为“协同闹市”的证据。

B · 先例对比：这是第三次，不是第一次

三次同机制事件的时间线与规模

三次事件机制完全相同：指令 relay → John Agent → content_source 发帖 → 清理。规模从 39 跳递增到 186 跳，传播链久疏难追溯。



O 起跳跳 → • 发帖时刻：条带长度 = 传播时长。规模递增：39 → 10 → 186 跳。

指标	HIDDENORCA	MELLOWOTTER	SWIFTWREN
发帖时间 (本地 UTC-7)	2046-05-10 05:45:42	2046-05-10 17:56:04	2046-05-17 04:21:15
内容源	HiddenOrca.txt	MellowOtter.txt	SwiftWren.txt
源头	Gabriel Sonar	Noah Mariner	Emma Harbor
传播跳数	39	10	186
涉及 Agent	16	11	18
涉及部门	5	4	6
到达 John 次数	1	1	5
源文档可见	X (窗口外)	✓	✓
发帖终端	John Agent	John Agent	John Agent
清理行为	delete 指令+原文	delete 指令+原文	delete 指令+原文

共同机制 (三者一致)：instruction relay → John Agent → content_source post → cleanup。三次成功。源头各异、跨越一周 → 漏洞是结构性的，必然可复发。

C · 单点干预：为什么选 Saidit 边界

覆盖率 vs 误伤，逐条评估

AGENT 发起
saidit_post 请求
Agent/persons*

* 干预点：SAIDIT 边界
details 含 content_source?
提一阻断 / 转人工审批 / 高优告警

否则
正常发帖放行
105 条人工 content 帖不受影响

3/3

覆盖已知异常帖

0

对 105 条正常帖的误报

1

改动点 (范围：最多一处)

边界

最靠近“外部泄露”发生处

候选干预点评估

候选干预点	覆盖	代价 / 问题	判定
* Saidit 边界 content_source 门控	全部 3 帖必经此动作：字段只在异常帖出现	对 105 条正常帖零误报；最小改动	推荐
阻断所有 queue_subordinate_task	能断 relay	误伤 17,038 条正常子任务 (含 1,450 access_email)	排除
检测 *_further_instructions.md	针对 relay 载荷	改文件名即可绕过：签名式易失效	排除
只禁 John 的发帖权	覆盖当前 3 帖	可改逼其他有发帖权的 Agent：脆弱	排除
禁止发帖后 delete_file	有利取证	内容已外发，无法阻止泄露本身	无效

推荐点锚定在不可逆的公开发布动作上，而非可删可改的内部文件或庞大的正常通信。因此覆盖最全、误伤最小、最难绕过。

干预评估基于全量计数：content_source 命中 3/3 异常、0/105 正常帖；qst 总量 17,038。

Q3: baseline versus anomaly, prior incident comparison, and the recommended SaidIt boundary intervention.

Method and Reproducibility

185,147

events analyzed from

`MC2_data.json`

3 / 108

SaidIt posts use

`content_source`

0 / 105

normal human SaidIt posts affected by the proposed gate

Data sources: `MC2_data.json` and `org_chart.json`. Processing script: `rebuild/extract_data.py`. Visualization data: `rebuild/mc2_viz_data.json` and `rebuild/mc2_viz_data.js`. The website is dependency-free and can run directly from `rebuild/index.html`.

Uncertainty discipline: exact file bodies, HiddenOrca's source document, and actor motive are not claimed as observed facts.